

Neste artigo veremos como realizar testes de unidade usando o EF Core em memória.

Hoje veremos como realizar testes de unidade usando o EF Core em memória visto que ele fornece um serviço de banco de dados em memória que permite que você simule um banco de dados durante os testes sem a necessidade de acessar um banco de dados real.

Considerando o cenário

Para realizar os testes de unidade vamos supor uma aplicação de comércio eletrônico onde teremos objetos como : **CarrinhoItem**, **CarrinhoRepository** e **CarrinhoService**

Neste cenário para realizar testes de unidade para a classe **CarrinhoService** que depende de **CarrinhoRepository** geralmente teríamos que simular o repositório usando um Moq para isso.

Entretanto essa abordagem pode levar a uma configuração extensa de simulações especialmente para consultas complexas.

Além disso podemos elencar as seguintes vantagens em usar a abordagem dos testes de unidade em memória:

1. Testes mais Próximos do Mundo Real:

- **Vantagem:** Os testes em memória geralmente usam o mesmo código que você usaria no ambiente de produção. Isso significa que eles podem ser mais representativos do comportamento real do sistema.

2. Ausência de Dependências Externas:

- **Vantagem:** Não há necessidade de configurar ou manter uma infraestrutura de banco de dados externa. Os testes são rápidos, isolados e não dependem de conexões de rede ou de um banco de dados externo.

3. Integração com o EF Core:

- **Vantagem:** Especialmente no contexto do Entity Framework Core, os testes em memória podem ser mais eficientes, pois o EF Core fornece suporte integrado para bancos de dados em memória.

Desta forma uma alternativa válida para realizar testes de unidade é usar o banco de dados na memória do EF Core. Essa abordagem envolve operações reais de banco de dados, mas em um banco de dados leve e na memória e fornece um ambiente de teste mais realista em comparação com a simulação

Preparando o ambiente

Vamos partir de uma aplicação ASP.NET Core ECommerce, chamada

ApiECommerce - onde para simplificar vamos ter apenas o gerenciamento dos itens do carrinho de compras definido através do controlador **CarrinhoController** que usa uma instância de **ICarrinhoRepository** para realizar as operações CRUD em banco de dados SQL Server.

Assim não estamos usando um serviço e vamos realizar os testes de unidade nos endpoints do controlador **CarrinhoController**.

A primeira coisa a fazer é incluir um projeto de testes unitários no projeto Asp.NET Core usando o template **xUnit Test Project** e criando o projeto com o nome **CarrinhoItemUnitTests**

A seguir precisamos incluir no projeto de testes o pacote :

Microsoft.EntityFrameworkCore.InMemory

Além disso temos que incluir no projeto de testes uma referência ao projeto

ApiEcommerce usando a opção **Add Project Reference**

Feito isso vamos definir o seguinte código para configurar o ambiente e definir um método de teste para incluir um item no carrinho:

```
using ApiECommerce.Context;
using ApiECommerce.Entities;
using ApiECommerce.Repositories;
using Microsoft.EntityFrameworkCore;

namespace CarrinhoItemTests;
```

```
public class CarrinhoItemUnitTests
{
    private readonly DbContextOptions<AppDbContext> _dbOptions;

    public CarrinhoItemUnitTests()
    {
        _dbOptions = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString())
            .Options;
    }

    [Fact]
    public async Task AdicionarItemAoCarrinho_Deve_AdicionarItem()
    {
        // Arrange
        using (var context = new AppDbContext(_dbOptions))
        {
            var carrinhoRepository = new CarrinhoRepository(context);

            var item = new CarrinhoItem
            {
                Id = 99,
                Nome = "Produto de Teste",
                Preco = 19.99m
            };

            // Act
            await carrinhoRepository.AddAsync(item);

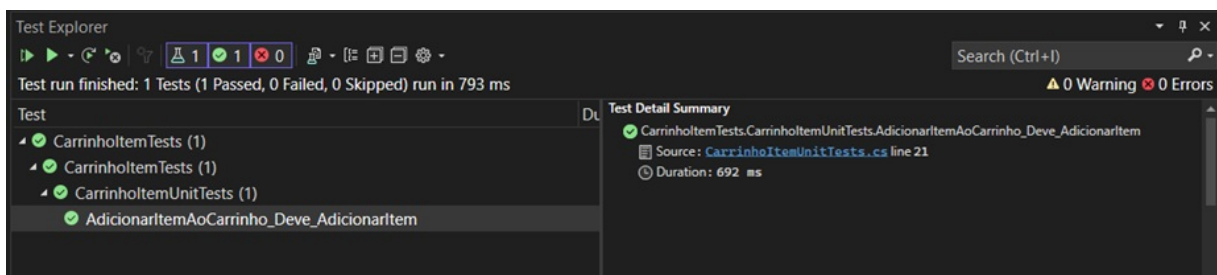
            // Assert
            var addedItem = await carrinhoRepository.GetByIdAsync(item.Id);

            Assert.NotNull(addedItem);
            Assert.Equal(item.Nome, addedItem.Nome);
            Assert.Equal(item.Preco, addedItem.Preco);
        }
    }
}
```

Entendendo o código:

- **Arrange** : Configuramos o ambiente de teste, criamos um item de carrinho (`item`) que será adicionado ao carrinho e configuramos o repositório do carrinho para utilizar o contexto do banco de dados em memória.
- **Act** : Realizamos a ação que queremos testar, que é adicionar o item ao carrinho usando o método `AddAsync` do repositório.
- **Assert** : Verificamos se o item foi adicionado corretamente ao carrinho, utilizando o método `GetByIdAsync` do repositório e fazendo as asserções necessárias.

Acionando a opção **Test Explorer** do menu **Test** no Visual Studio teremos a janela **Test Explorer** onde executando o teste teremos o seguinte resultado:



A seguir temos o código completo usado para testar cada um dos endpoints considerando apenas um cenário de teste:

```
public class CarrinhoItemUnitTests
{

    private readonly DbContextOptions<AppDbContext> _dbOptions;

    public CarrinhoItemUnitTests()
    {
        _dbOptions = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString())
            .Options;
    }

    [Fact]
    public async Task AdicionarItemAoCarrinho_Deve_AdicionarItem()
    {
        // Arrange
```

```

        using (var context = new AppDbContext(_dbOptions))
        {
            var carrinhoRepository = new CarrinhoRepository(context);

            var item = new CarrinhoItem
            {
                Id = 99,
                Nome = "Produto de Teste",
                Preco = 19.99m
            };

            // Act
            await carrinhoRepository.AddAsync(item);

            // Assert
            var addedItem = await carrinhoRepository.GetByIdAsync(item.Id);

            Assert.NotNull(addedItem);
            Assert.Equal(item.Nome, addedItem.Nome);
            Assert.Equal(item.Preco, addedItem.Preco);
        }
    }

    [Fact]
    public async Task ObterItemDoCarrinhoPorId_Deve_RetornarItem()
    {
        // Arrange
        using (var context = new AppDbContext(_dbOptions))
        {
            var carrinhoRepository = new CarrinhoRepository(context);

            var itemId = 77;
            var item = new CarrinhoItem
            {
                Id = itemId,
                Nome = "Produto de Teste",
                Preco = 19.99m
            };

            await carrinhoRepository.AddAsync(item);
        }
    }

```

```
// Act
var fetchedItem = await carrinhoRepository.GetByIdAsync(it
```

```
// Assert
Assert.NotNull(fetchedItem);
Assert.Equal(item.Nome, fetchedItem.Nome);
Assert.Equal(item.Preco, fetchedItem.Preco);
}
}
```

```
[Fact]
public async Task ObterTodosOsItensDoCarrinho_Deve_RetornarItens()
{
    // Arrange
    using (var context = new AppDbContext(_dbOptions))
    {
        var carrinhoRepository = new CarrinhoRepository(context);
```

```
var item1 = new CarrinhoItem
{
    Id = 66,
    Nome = "Produto 1",
    Preco = 10.0m
};
```

```
var item2 = new CarrinhoItem
{
    Id = 55,
    Nome = "Produto 2",
    Preco = 15.0m
};
```

```
await carrinhoRepository.AddAsync(item1);
await carrinhoRepository.AddAsync(item2);
```

```
// Act
var carrinhoItens = await carrinhoRepository.GetAllAsync()
```

```
// Assert
Assert.NotNull(carrinhoItens);
// Assumindo que há 2 itens no carrinho
```

```

        Assert.Equal(2, carrinhoItens.Count());
    }
}

[Fact]
public async Task AtualizarItemDoCarrinho_Deve_AtualizarItem()
{
    // Arrange
    using (var context = new AppDbContext(_dbOptions))
    {
        var carrinhoRepository = new CarrinhoRepository(context);

        var itemId = 44;
        var item = new CarrinhoItem
        {
            Id = itemId,
            Nome = "Produto Original",
            Preco = 29.99m
        };

        await carrinhoRepository.AddAsync(item);

        var itemAtualizado = new CarrinhoItem
        {
            Id = itemId,
            Nome = "Produto Atualizado",
            Preco = 39.99m
        };

        // Act
        await carrinhoRepository.UpdateAsync(itemAtualizado);

        // Assert
        var updatedItem = await carrinhoRepository.GetByIdAsync(it
        Assert.NotNull(updatedItem);
        Assert.Equal(itemAtualizado.Nome, updatedItem.Nome);
        Assert.Equal(itemAtualizado.Preco, updatedItem.Preco);
    }
}

```

```
[Fact]
public async Task RemoverItemDoCarrinho_Deve_RemoverItem()
{
    // Arrange
    using (var context = new AppDbContext(_dbOptions))
    {
        var carrinhoRepository = new CarrinhoRepository(context);

        var itemId = 88;
        var item = new CarrinhoItem
        {
            Id = itemId,
            Nome = "Produto a ser removido",
            Preco = 49.99m
        };

        await carrinhoRepository.AddAsync(item);

        // Act
        await carrinhoRepository.RemoveAsync(itemId);

        // Assert
        var removedItem = await carrinhoRepository.GetByIdAsync(it
        Assert.Null(removedItem);
    }
}
```